

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)



Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа № 6
по дисциплине: Вычислительная математика
по теме: Численное интегрирование

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: ст. преподаватель:
Четвертухин В. Р.

Белгород 2024

Цель работы: Изучить основные численные формулы интегрирования, особенности их алгоритмизации.

Цель работы обуславливает постановку и решение следующих задач:

- 1) Рассмотреть теоретические основы численного интегрирования для аппроксимации разных порядков.
- 2) Научиться выбирать формулы интегрирования и алгоритмизировать их в зависимости от численной ситуации с вниманием к проблемам точности, численной стабильности и релевантности поставленной задачи.
- 3) Выполнить индивидуальное задание, закрепляющее на практике полученные знания и практические навыки (номер задания соответствует номеру студента по журналу; если этот номер больше, чем максимальное число заданий, тогда вариант задания вычисляется по формуле: номер по журналу % максимальный номер задания, где % — остаток от деления).

Алгоритм выполнения индивидуального задания находится в разделе «Ход выполнения практической части лабораторной работы».

- 4) Отразить в отчете все полученные результаты, включая графики (при необходимости), тексты программ. Сделать выводы.

Вариант №7

7.
$$f(x) = \frac{\cosh(1.1x) \cdot \log(1.2+x)}{x^2+1.3}$$

Код программы:

```
%%writefile lab6_my_exercise.rs
fn ln(x: f64) -> f64 {
    x.ln()
}
fn cos(x: f64) -> f64 {
    x.cos()
}

fn sin(x: f64) -> f64 {
    x.sin()
}

fn f(x: f64) -> f64 {
    (cos(1.1 * x) * ln(1.2 + x)) / (x.powi(2) + 1.3)
}

fn left_rectangle_method(a: f64, b: f64, n: usize) -> (f64, f64) {
    let h = (b - a) / (n as f64);
    let mut result: f64 = 0.0;

    for i in 0..n {
        let xi = a + (i as f64) * h;
        result += f(xi);
    }

    result *= h;

    let max_derivative = 0.064;
    let error = (max_derivative * (b - a) * h) / 2.0;
    (result, error)
}

fn right_rectangle_method(a: f64, b: f64, n: usize) -> (f64, f64) {
    let h = (b - a) / (n as f64);
    let mut result: f64 = 0.0;

    for i in 1..n {
        let xi = a + (i as f64) * h;
        result += f(xi);
    }

    result *= h;

    let max_derivative = 0.064;
    let error = (max_derivative * (b - a) * h) / 2.0;
    (result, error)
}

fn midpoint_rectangle_method(a: f64, b: f64, n: usize) -> (f64, f64) {
```

```

    let h = (b - a) / (n as f64);
    let mut result: f64 = 0.0;

    for i in 0..n {
        let xi = a + (i as f64) * h + h / 2.0;
        result += f(xi);
    }

    result *= h;

    let max_derivative = 3.0298851806662115;
    let error = (max_derivative * (b - a) * h * h) / 24.0;
    (result, error)
}

fn trapezoid_method(a: f64, b: f64, n: usize) -> (f64, f64) {
    let h = (b - a) / (n as f64);
    let mut result: f64 = 0.0;

    for i in 1..n {
        let xi = a + (i as f64) * h;
        result += f(xi);
    }

    result += (f(a) + f(b)) / 2.0;
    result *= h;

    let error = (b - a) * (h.powi(2)) / 12.0;
    (result, error)
}

fn simpsons_method(a: f64, b: f64, n: usize) -> (f64, f64) {
    let h = (b - a) / (n as f64);
    let mut result = f(a) + f(b);

    for i in 1..n {
        let xi = a + (i as f64) * h;
        result += if i % 2 == 0 { 2.0 * f(xi) } else { 4.0 * f(xi) };
    }

    result *= h / 3.0;

    let error = (b - a) * (h.powi(4)) / 180.0;
    (result, error)
}

fn main() {
    let a = 0.0;
    let b = 10.0;
    let n = 100;

```

```
let (lrt_method, lrt_error) = left_rectangle_method(a, b, n);
let (rrt_method, rrt_error) = right_rectangle_method(a, b, n);
let (rt_method, rt_error) = midpoint_rectangle_method(a, b, n);
let (tp_method, tp_error) = trapezoid_method(a, b, n);
let (s_method, s_error) = simpsons_method(a, b, n);

println!("Приближенное значение интеграла методом левых
прямоугольников = {}", lrt_method);
println!("Оценка погрешности метода левых прямоугольников (абсолютная
погрешность): {}", lrt_error);
println!("Оценка погрешности метода левых прямоугольников
(относительная погрешность в процентах): {}%\n", (lrt_error / lrt_method)
* 100.0);

println!("Приближенное значение интеграла методом правых
прямоугольников = {}", rrt_method);
println!("Оценка погрешности метода правых прямоугольников (абсолютная
погрешность): {}", rrt_error);
println!("Оценка погрешности метода правых прямоугольников
(относительная погрешность в процентах): {}%\n", (rrt_error / rrt_method)
* 100.0);

println!("Приближенное значение интеграла методом прямоугольников (1
порядка) = {}", rt_method);
println!("Оценка погрешности метода прямоугольников (абсолютная
погрешность): {}", rt_error);
println!("Оценка погрешности метода прямоугольников (относительная
погрешность в процентах): {}%\n", (rt_error / rt_method) * 100.0);

println!("Приближенное значение интеграла методом трапеций (2 порядка)
= {}", tp_method);
println!("Оценка погрешности метода трапеций (абсолютная погрешность):
{}", tp_error);
println!("Оценка погрешности метода трапеций (относительная
погрешность в процентах): {}%\n", (tp_error / tp_method) * 100.0);

println!("Приближенное значение интеграла методом Симпсона (4 порядка)
= {}", s_method);
println!("Оценка погрешности метода Симпсона (абсолютная погрешность):
{}", s_error);
println!("Оценка погрешности метода Симпсона (относительная
погрешность в процентах): {}%\n", (s_error / s_method) * 100.0);
}
```

Результаты работы

```
Приближенное значение интеграла методом левых прямоугольников = 0.04719606045344793
Оценка погрешности метода левых прямоугольников (абсолютная погрешность): 0.032
Оценка погрешности метода левых прямоугольников (относительная погрешность в процентах): 67.80226928381737%

Приближенное значение интеграла методом правых прямоугольников = 0.033171325315451416
Оценка погрешности метода правых прямоугольников (абсолютная погрешность): 0.032
Оценка погрешности метода правых прямоугольников (относительная погрешность в процентах): 96.46886187298098%

Приближенное значение интеграла методом прямоугольников (1 порядка) = 0.04095862304443849
Оценка погрешности метода прямоугольников (абсолютная погрешность): 0.012624521586109216
Оценка погрешности метода прямоугольников (относительная погрешность в процентах): 30.82262207011234%

Приближенное значение интеграла методом трапеций (2 порядка) = 0.040188970330031076
Оценка погрешности метода трапеций (абсолютная погрешность): 0.008333333333333335
Оценка погрешности метода трапеций (относительная погрешность в процентах): 20.73537407129408%

Приближенное значение интеграла методом Симпсона (4 порядка) = 0.04070439296415661
Оценка погрешности метода Симпсона (абсолютная погрешность): 0.00000555555555555558
Оценка погрешности метода Симпсона (относительная погрешность в процентах): 0.013648540491557407%
```

Вывод: Мы изучили основные численные формулы интегрирования и особенности их алгоритмизации.